Cloud Computing Option 2: Serverless Image Processing Application on AWS Lambda

Luis Sanchez Juan Garcia Edward King

June 2026

Sapienza University of Rome Cloud Computing

[EDDIE: Update abstract after completing the report]

Abstract

This report presents the design, implementation, deployment, and performance evaluation of a serverless image processing application developed using Amazon Web Services (AWS). The application stores input images in Amazon S3 and performs image resizing operations through AWS Lambda functions invoked via an HTTP API exposed by Amazon API Gateway. The deployment uses Amazon API Gateway, AWS Lambda, Amazon S3, and Amazon CloudWatch.

The main objective of the project is to evaluate the performance and scalability of a serverless deployment under different workload conditions. Experimental tests were conducted using multiple image sizes and varying levels of concurrency. Metrics including response time, throughput, invocation count, execution duration, concurrent executions, error rate, and throttling behaviour were collected and analysed using Apache JMeter and Amazon CloudWatch.

The results show that AWS Lambda is able to scale automatically in response to increasing workload intensity while maintaining acceptable performance levels under the tested scenarios. A cost analysis is also presented, comparing the deployed serverless solution with an alternative EC2-based deployment over a 6-month operational period.

Contents

1 Introduction

Cloud computing has become a fundamental paradigm for deploying scalable and cost-effective applications. Among the various cloud service models, serverless computing has gained significant attention due to its ability to automatically allocate resources according to workload demands while eliminating the need for infrastructure management. Amazon Web Services (AWS) Lambda is one of the most widely adopted serverless platforms, enabling developers to execute application logic without provisioning or maintaining servers.

[EDDIE: Probably have to add some references here] Image processing applications represent a suitable use case for serverless architectures because workloads can fluctuate considerably over time. Traditional server-based deployments often require resources to be provisioned for peak demand, resulting in underutilised infrastructure during periods of low activity. In contrast, serverless deployments dynamically allocate resources according to demand, potentially reducing operational costs while maintaining acceptable performance levels.

[LUIS: I made some changes to the text below, can you check that it's correct?] This project investigates the deployment of a serverless image processing application using AWS cloud services. The application enables users to upload images through an HTTP API, store them in Amazon S3, and perform image resizing operations using AWS Lambda functions. The deployment integrates Amazon API Gateway, AWS Lambda, Amazon S3, and Amazon CloudWatch to provide a scalable and observable cloud-native architecture.

[LUIS: ALso check the text below, I made some changes to it] The primary focus of the project is not the image processing functionality itself, but rather the evaluation of the performance and scalability characteristics of the serverless deployment. Experimental testing was conducted under varying workload conditions to determine how effectively the system scales in response to increasing demand and to identify potential performance limitations or bottlenecks.

1.1 Project Objectives

The objectives of this project are:

- To design and implement a serverless image processing application using AWS services.
- To deploy the application using AWS Lambda and API Gateway.
- To evaluate the performance of the deployment under different workload conditions.
- To analyse the scalability characteristics of serverless computing.
- To investigate the relationship between workload intensity and resource utilisation.
- To estimate the operational cost of the deployment and compare it with an alternative cloud deployment model.

1.2 Project Scope

[LUIS: I made some changes to the text below, can you check that it's correct?] The developed application provides a simple image processing workflow consisting of image upload, storage, and resizing operations. Users interact with the system through HTTP endpoints exposed by Amazon API Gateway. Uploaded images are stored in Amazon S3, while image processing tasks are executed by AWS Lambda functions.

The scope of the project includes:

[LUIS: Here also changes]

- Image upload through HTTP API endpoints.

- Image storage using Amazon S3.
- Image resizing operations executed by AWS Lambda.
- Monitoring and metric collection through Amazon CloudWatch.
- Experimental performance and scalability evaluation.

The project does not focus on advanced image analysis, machine learning functionality, or front-end application development. Instead, the emphasis is placed on evaluating the behaviour of the serverless infrastructure under controlled workload conditions.

1.3 Purpose and Scope of the Evaluation

The purpose of the performance evaluation is to assess the scalability, responsiveness, and reliability of the proposed serverless image processing application under different workload conditions.

The study aims to determine whether the AWS Lambda-based deployment is capable of automatically scaling in response to increasing demand while maintaining an acceptable level of service. In particular, the evaluation investigates how workload intensity affects response time, throughput, concurrency levels, and overall system behaviour.

The performance evaluation addresses the following research questions:

[EDDIE: Check that these questions have been answered in the report]

- Does the application scale effectively as workload intensity increases?
- How does response time vary under different levels of concurrency?
- What throughput can be achieved under increasing load?
- How many concurrent Lambda executions are required to maintain performance?
- Are there any observable bottlenecks within the processing workflow?
- Does the deployment maintain reliable operation under sustained and burst workloads?

[LUISJUAN: Do we have all of the metrics below?] The evaluation considers both user-oriented and system-oriented performance metrics. User-oriented metrics include response time, throughput, availability, and error rate, measured primarily from the client side using Apache JMeter. System-oriented metrics include Lambda invocation count, execution duration, concurrent executions, errors, throttling behaviour, and overall scalability characteristics, collected through Amazon CloudWatch.

Experimental data were collected using Amazon CloudWatch and the selected load generation tools. The resulting measurements provide quantitative evidence regarding the performance and scalability of the serverless deployment and support the conclusions presented later in this report.

2 System Design

2.1 Overall Architecture

The proposed solution follows a serverless architecture built entirely using managed AWS services. The architecture was designed to provide automatic scalability, low operational overhead, and seamless integration between storage, processing, and monitoring components.

Figure **??** illustrates the overall system architecture.

The system is based on Amazon API Gateway, AWS Lambda, Amazon S3, and Amazon CloudWatch. Input images are stored directly in an Amazon S3 input bucket, where they are organised by workload category using the prefixes `small/`, `medium/`, and `large/`. A client then sends an HTTP request to Amazon API Gateway through the `/process` endpoint. API Gateway invokes an AWS Lambda function, which retrieves the selected image from the S3 input bucket, performs the requested resizing operation, and stores the processed result in an Amazon S3 output bucket. Amazon CloudWatch collects execution logs and monitoring metrics such as invocations, duration, concurrent executions, errors, and throttles.

The image processing workflow consists of the following steps:

1. A client selects an image already stored in the S3 input bucket.
2. The client submits an image processing request through the `/process` HTTP endpoint.
3. Amazon API Gateway receives the request and forwards it to the processing Lambda function.
4. The Lambda function reads the input image from Amazon S3.
5. The Lambda function performs the requested resizing operation.
6. The processed image is written to the S3 output bucket.
7. Amazon CloudWatch records execution logs and performance metrics during the request lifecycle.

This architecture eliminates the need to provision or manage virtual machines while allowing execution resources to scale automatically in response to workload fluctuations. Short Version: Client → API Gateway → Lambda → S3 Input / S3 Output → CloudWatch

Client → API Gateway
API Gateway → Lambda
Lambda → S3 Input Bucket (read image)
Lambda → S3 Output Bucket (write processed image)
Lambda / API Gateway → CloudWatch (logs and metrics)
The image processing workflow consists of the following steps:

1. A client submits an image processing request through a REST API endpoint.
2. AWS API Gateway receives the request and performs request routing.
3. The request is forwarded to an AWS Lambda function responsible for image processing.
4. The Lambda function retrieves or receives the image data and performs the required resizing operation.
5. Input and processed images are stored within Amazon S3.
6. The processed image or processing result is returned to the client through API Gateway.
7. Amazon CloudWatch continuously records operational and performance metrics generated during execution.

[EDDIE: Once that flow diagram is made make sure the paragraph below makes sense] This architecture eliminates the need to provision or manage virtual machines while allowing resources to scale automatically in response to workload fluctuations.

2.2 AWS Components

The deployment relies on four primary AWS services: API Gateway, AWS Lambda, Amazon S3, and Amazon CloudWatch. Together, these services provide the functionality required to implement, operate, and evaluate the serverless image processing application.

2.2.1 API Gateway

Amazon API Gateway provides the external interface through which clients interact with the application. In the final version of the system, API Gateway exposes a single HTTP endpoint, `/process`, which is used to trigger the image-processing Lambda function.

API Gateway acts as the entry point of the system, receiving incoming HTTP requests and forwarding them to AWS Lambda. Each request specifies the location of the image in Amazon S3 together with the required processing parameters, such as the resizing operation and output dimensions. Since images are stored directly in S3 before processing, the API layer remains lightweight and focuses only on request routing and backend integration.

This design simplifies the architecture, reduces the number of application components, and allows performance testing to concentrate on the critical processing path formed by API Gateway, Lambda, and Amazon S3.

2.2.2 AWS Lambda

AWS Lambda provides the serverless execution environment responsible for performing image processing operations.

When a request is received from API Gateway, the corresponding Lambda function is invoked automatically. The function performs the required image resizing operation and stores the resulting image within Amazon S3.

One of the primary advantages of AWS Lambda is its ability to automatically allocate execution resources according to workload demand. This behaviour makes Lambda particularly suitable for workloads characterised by unpredictable traffic patterns and varying request volumes.

The Lambda configuration used during experimentation is summarised in Table **??**.

[LUIS CORRECTION]

Table 1: Deployment Configuration

Parameter	Value
Lambda Runtime	Python 3.11
Memory Allocation	128 MB
Timeout	3 seconds
Region	us-east-1 (US East / N. Virginia)

2.2.3 Amazon S3

[LUIS CORRECTION] Amazon S3 was organised into an input bucket and an output bucket. The input bucket stored the original images grouped by workload category using the prefixes `small/`, `medium/`, and `large/`. The output bucket stored processed images using an operation-based naming convention such as `processed/resize/`, `processed/grayscale/`, and `processed/resize_grayscale/`. This structure simplified workload preparation, result verification, and performance testing across different image sizes.

[LUIS CORRECTION] if you want a table:

Table 2: S3 Bucket Structure

Bucket / Prefix	Purpose
Input bucket / `small/`	Small input images
Input bucket / `medium/`	Medium input images
Input bucket / `large/`	Large input images
Output bucket / `processed/resize/`	Resized output images
Output bucket / `processed/grayscale/`	Grayscale output images
Output bucket / `processed/resize_grayscale/`	Resized grayscale output images

2.2.4 Amazon CloudWatch

Amazon CloudWatch is used to monitor system behaviour and collect performance metrics throughout experimental testing.

CloudWatch provides visibility into both operational and performance-related aspects of the deployment. Metrics collected during experimentation include invocation count, execution duration, concurrent executions, error count, and throttling events.

These measurements form the basis of the performance evaluation presented later in this report and enable the analysis of system scalability under varying workload conditions.

[JUAN: Insert screenshots or metric dashboard references if required.]

2.3 Features Under Evaluation

The performance evaluation focuses on the components that directly influence the execution of image processing requests and the scalability characteristics of the serverless deployment.

The primary components under evaluation are:

- AWS Lambda image processing functions.
- AWS API Gateway request handling.
- Amazon S3 storage interactions.

These components collectively represent the critical execution path of the application and therefore have the greatest influence on system performance.

The study focuses specifically on the behaviour of the serverless processing pipeline under varying workload conditions. Particular attention is given to response time, throughput, concurrency levels, execution duration, and scaling behaviour.

Several components are intentionally excluded from the evaluation:

- User authentication and authorisation services.
- Front-end interfaces and client-side rendering.
- External network conditions beyond the AWS environment.
- Advanced image processing algorithms unrelated to system scalability.

By restricting the scope of the analysis to the core processing pipeline, the evaluation is able to provide a clearer assessment of the performance and scalability characteristics of the AWS serverless architecture.

The resulting experimental study therefore focuses on determining whether AWS Lambda can efficiently scale image processing workloads while maintaining acceptable levels of responsiveness, throughput, and reliability.

3 Implementation

3.1 Processing Service

The processing service performs image resizing operations on images stored in Amazon S3.

Endpoint

`/process`

Example Request

```
{
  "bucket": "bucket-name",
  "key": "small/image.jpg",
  "operation": "resize",
  "width": 256,
  "height": 256
}
```

3.2 Data Preparation

Before experimental testing, the input images were uploaded directly to the Amazon S3 input bucket. To support workload variation, the dataset was organised into three categories:

- `small/` for small images
- `medium/` for medium images
- `large/` for large images

Processed images were stored in the output bucket using operation-specific prefixes such as `processed/resize/`, `processed/grayscale/`, and `processed/resize_grayscale/`.

3.3 Deployment

The application was deployed within AWS using the following managed services:

- Amazon API Gateway
- AWS Lambda
- Amazon S3
- Amazon CloudWatch

Deployment configuration parameters are summarised in Table **??**.

Table 3: Deployment Configuration

Parameter	Value
Lambda Runtime	Python 3.11
Memory Allocation	128 MB
Timeout	3 seconds
Region	us-east-1 (US East / N. Virginia)

4 Experimental Methodology

4.1 Evaluation Objectives

The evaluation focuses on the following metrics:

- Response Time
- Throughput
- Scalability
- Concurrent Executions
- Error Rate

4.2 Workload Design

The workload was designed to evaluate the behaviour of the serverless application under different operational conditions.

Requests consisted of image resizing operations performed through the REST API exposed by API Gateway.

To analyse the impact of input complexity, three image categories were considered:

- Small images
- Medium images
- Large images

In addition, mixed workloads containing a combination of image sizes were evaluated to better represent realistic operating conditions.

4.3 Workload Profile

Each experiment followed a workload profile designed to observe both scale-out and scale-in behaviour.

The workload consisted of five phases:

1. Warm-up (WU)
2. Ramp-up (RU)
3. Steady-State (SS)
4. Ramp-down (RD)
5. Cool-down (CD)

During the warm-up phase, a small number of requests were generated to initialise the Lambda execution environment and reduce cold-start effects.

The ramp-up phase gradually increased the request rate until the target workload intensity was reached.

The steady-state phase maintained a constant workload to observe stable system behaviour.

The ramp-down phase gradually reduced the workload to evaluate the ability of the deployment to scale in.

Finally, the cool-down phase allowed the system to return to idle conditions.

Figure ?? illustrates the workload profile used during testing.

Figure 1: Workload profile used during performance testing.

4.4 Burst Workload Evaluation

In addition to the continuous workload profile, burst workload tests were performed to evaluate the ability of the deployment to respond to sudden increases in demand.

The burst workload consisted of a rapid increase in concurrent requests, followed by a return to normal operating conditions.

This experiment was designed to evaluate:

- Lambda scaling responsiveness.
- Temporary response time degradation.
- Concurrency growth behaviour.
- Recovery time after workload reduction.

4.5 Workload Scenarios

Four workload levels were considered.

Table 4: Workload Scenarios

Scenario	Concurrent Users
Small	10
Medium	50
Heavy	100
Stress	Until saturation

4.6 Testing Tools

The following tools were used:

- Apache JMeter
- AWS CloudWatch
- Postman

4.7 Metrics Collected

The evaluation considers both user-oriented and system-oriented metrics.

User-Oriented Metrics

- Average Response Time
- Throughput (requests/second)
- Error Rate
- Availability

System-Oriented Metrics

- Lambda Invocation Count
- Lambda Duration
- Concurrent Executions
- Throttling Events
- Scalability Behaviour

Metrics were collected using AWS CloudWatch and Apache JMeter.

4.8 Experimental Procedure

The experimental evaluation was designed to analyse the behaviour of the serverless image processing pipeline under different workload conditions. The tests focused on the `/process` endpoint, which triggered the Lambda-based resizing operation on images stored in Amazon S3.

Three image categories were considered in order to evaluate the impact of input size on system performance:

- Small images

- Medium images
- Large images

For each image category, three workload levels were tested:

- 10 concurrent users
- 50 concurrent users
- 100 concurrent users

Each experimental scenario was repeated three times under identical conditions in order to reduce the effect of transient fluctuations and obtain more representative measurements. In addition to the main workload matrix, separate stress tests were performed with 400 and 800 concurrent users to observe behaviour under substantially higher load.

Apache JMeter was used as the workload generation tool. For each run, JMeter sent HTTP POST requests to the API Gateway endpoint, while Amazon CloudWatch collected server-side metrics from the Lambda deployment. The main client-side metrics recorded by JMeter were response time, throughput, success rate, and error rate. The main server-side metrics collected through CloudWatch were invocation count, execution duration, concurrent executions, errors, and throttles.

Before each run, the JMeter listeners were cleared to avoid mixing results from different experiments. After each run, the output values were recorded, and average results were computed across the three repetitions of each scenario.

An additional continuous workload experiment was also attempted using a progressive increase/decrease load profile. However, that experiment could not be completed because the AWS Learner Lab account was automatically deactivated after excessive resource consumption. The partial graphs obtained from that run are therefore treated as qualitative supporting evidence rather than as a complete quantitative result.

5 Experimental Results

5.1 Response Time Analysis

Figure **??** shows the relationship between workload intensity and response time.

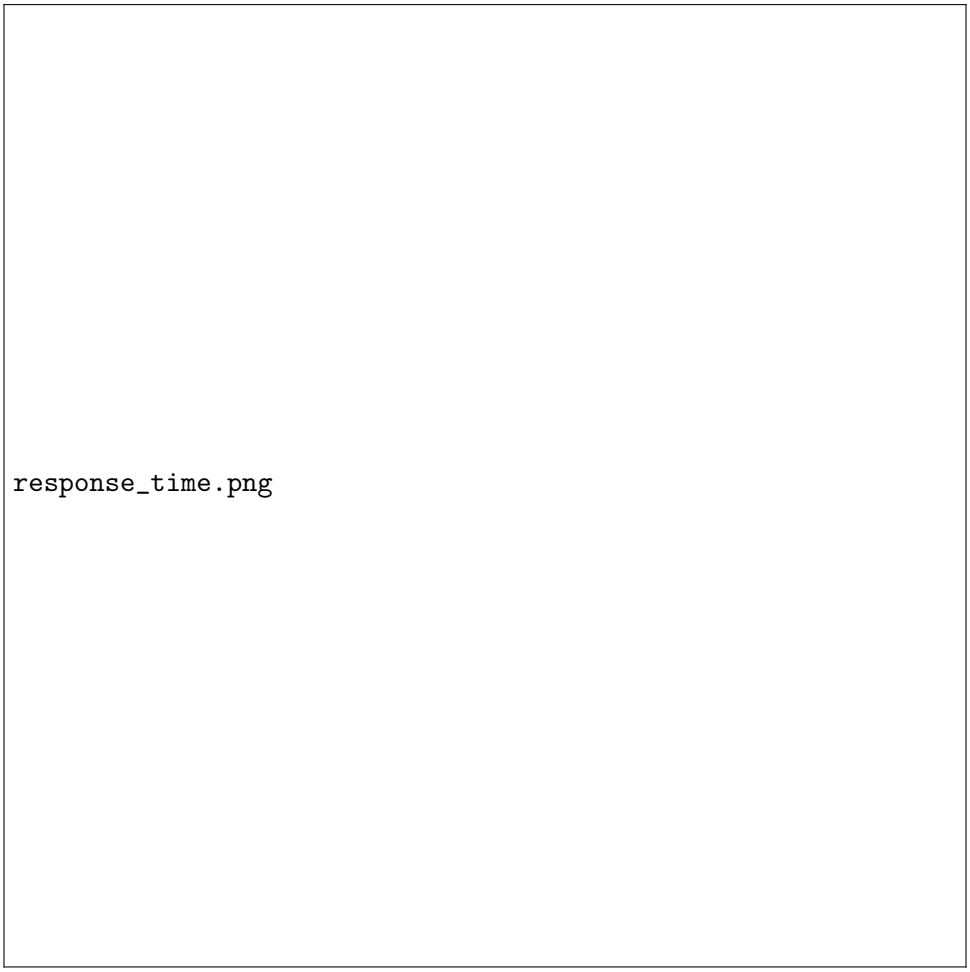

Figure 2: Response Time vs Concurrent Users

Discussion of observed trends goes here.

5.2 Throughput Analysis

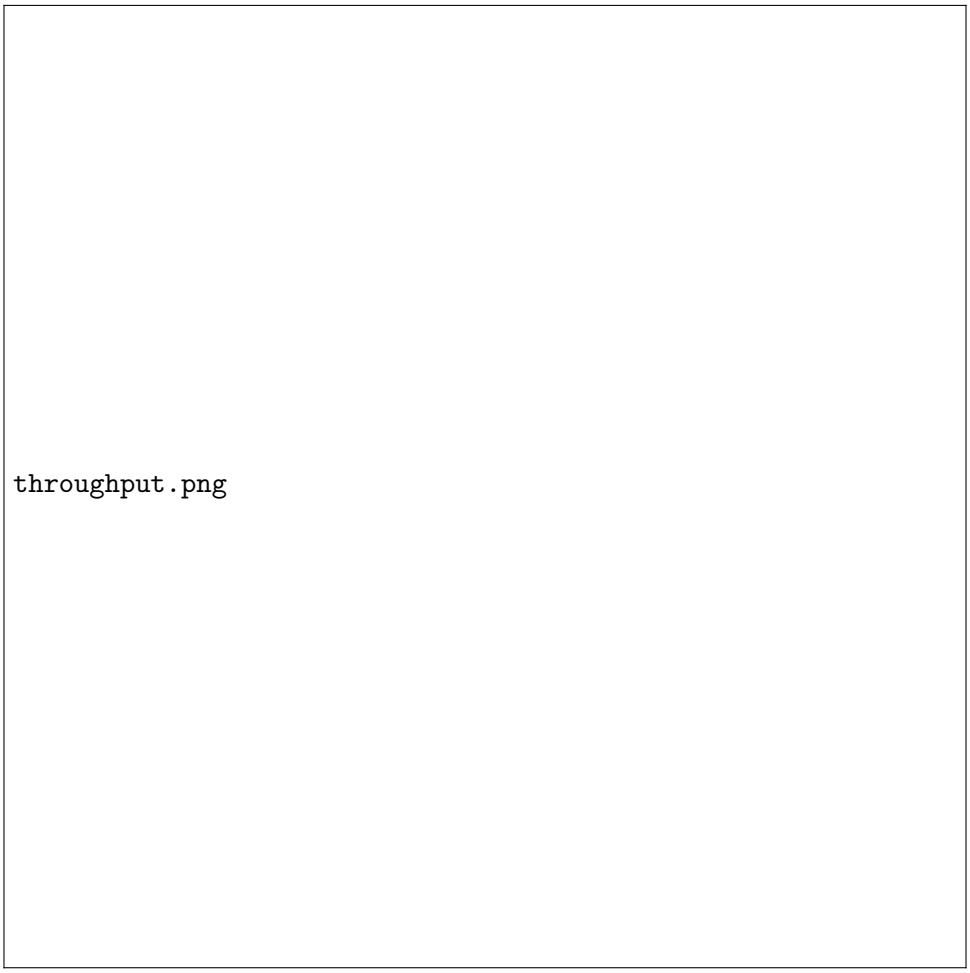

Figure 3: Throughput vs Concurrent Users

Discussion of throughput behavior goes here.

5.3 Concurrency Analysis

Figure 4: Concurrent Lambda Executions

CloudWatch observations and discussion go here.

5.4 Scalability Evaluation

This section discusses:

- Automatic scaling behavior.
- Maximum observed concurrency.
- Performance bottlenecks.
- Resource utilization.

5.5 Bottleneck Analysis

Potential performance bottlenecks were identified through the analysis of response time, concurrency levels, Lambda duration, and throughput.

The objective is to determine whether performance degradation originates from:

- Lambda execution limits.

- Memory allocation constraints.
- API Gateway limitations.
- Amazon S3 interaction delays.

Observed bottlenecks and their impact on system scalability are discussed using the experimental results.

5.6 Error Analysis

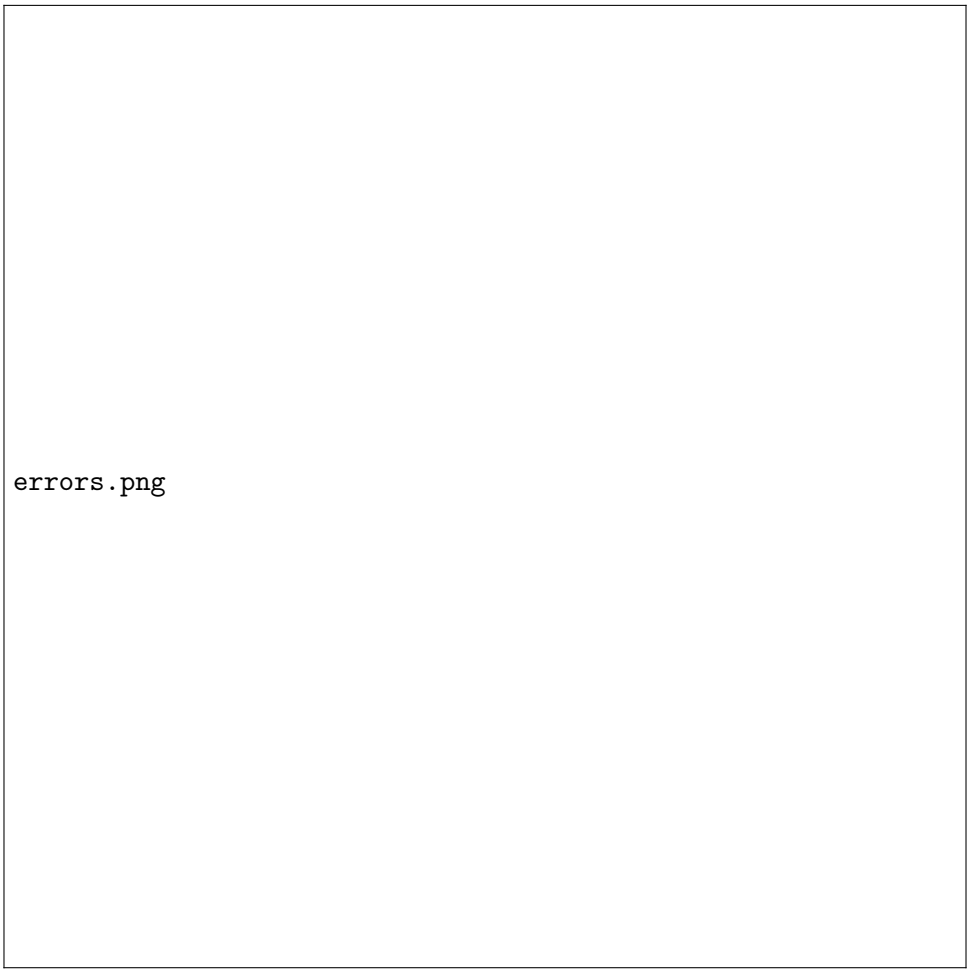

Figure 5: Error Rate vs Concurrent Users

Discussion of system reliability under load.

5.7 Availability Analysis

System availability was evaluated throughout all experimental runs.

Availability was calculated as:

$$Availability = \frac{Successful\ Requests}{Total\ Requests}$$

or equivalently

$$Availability = 1 - ErrorRate$$

The results indicate the ability of the deployment to maintain service continuity during increasing workload conditions.

5.8 Workload Verification

Before analysing performance metrics, the workload generated by JMeter was compared with the workload observed by AWS CloudWatch.

This verification ensures that the request rate generated by the load generator matches the workload effectively received by the system.

The comparison confirms that network limitations or configuration issues did not significantly alter the intended workload profile.

6 Experimental Limitations

Several factors may influence the obtained results:

- AWS Learner Lab resource limitations.
- Variability in cloud resource allocation.
- Internet latency fluctuations.
- Limited workload intensity imposed by AWS usage policies.
- Limited number of repetitions due to budget constraints.

Despite these limitations, the collected results provide a representative view of the scalability and performance characteristics of the deployment.

7 Cost Evaluation

7.1 Cost Assumptions

The cost comparison was performed under the same workload assumptions for both deployment alternatives. The estimation used the `us-east-1` region and assumed a workload of 10,000 requests per month together with a small amount of persistent S3 storage. This allowed a fair comparison between the deployed serverless solution and an equivalent EC2-based alternative providing the same functionality.

7.2 Serverless Deployment Cost

The deployed architecture was based on Amazon API Gateway, AWS Lambda, and Amazon S3. According to the AWS Pricing Calculator estimates produced for this project, the serverless solution has an estimated monthly cost of 0.16 USD, corresponding to 0.96 USD over a 6-month operational period.

7.3 Alternative EC2-Based Deployment Cost

As a comparison baseline, an alternative deployment was considered using one Amazon EC2 instance, attached block storage, and Amazon S3. This alternative represents a traditional always-on virtual-machine-based architecture capable of providing the same image processing functionality. The estimated monthly cost of this solution is 15.42 USD, corresponding to 92.52 USD over a 6-month operational period.

Table 5: Estimated 6-Month Cost Comparison

Solution	Monthly Cost (USD)	6-Month Cost (USD)
Lambda + API Gateway + S3	0.16	0.96
EC2 + EBS + S3	15.42	92.52

7.4 Discussion

The cost comparison shows that, under the assumed workload, the serverless architecture is significantly more cost-efficient than the EC2-based alternative. The Lambda-based deployment follows a pay-per-use model, which makes it well suited for workloads that do not require permanently allocated compute capacity. In contrast, the EC2-based alternative requires an always-on virtual machine and persistent storage, leading to a substantially higher fixed operational cost over the same 6-month period.

Beyond cost, the serverless architecture also reduces management effort because it does not require instance provisioning, patching, or capacity management. The EC2-based alternative may still be appropriate for workloads that are highly predictable and continuous, but for the workload considered in this project, the serverless solution offers a clear economic advantage.

8 Discussion

The experimental results indicate:

- Whether the system scales effectively.
- The workload levels at which degradation begins.
- The effectiveness of serverless deployment.
- Observed limitations and bottlenecks.

9 Conclusion

This project demonstrated the deployment of a serverless image processing application using AWS Lambda. Experimental evaluation showed how the system behaves under increasing workload intensity and highlighted the scalability characteristics of serverless computing. The results collected through CloudWatch and JMeter provide quantitative evidence of system performance. Finally, cost analysis demonstrated the economic implications of adopting a serverless architecture compared to traditional deployment solutions.

References

[1] Amazon Web Services Documentation. https://docs.aws.amazon.com

[2] Apache JMeter Documentation. https://jmeter.apache.org

[3] Cloud Computing Course Material.